

Lab Performance evaluation
Implementation of Gray Level Transformations
and Implementation of Image Histogram and Equalization

Digital Image Processing



Submitted By:

[Huzaifa Shahbaz]

[21-CE-009]

[Aqsa Shah]

[21-CE-039]

Submitted to:

Engr. Hareem Khan

Dated:

29th Oct, 2024

Department of Computer Engineering,
HITEC University, Taxila

Lab Task No 1:

Solution:

Brief description (3-5 lines)

Enhancing Low-Light Images for Security Footage
--

: You are working with security footage taken at night in low-light conditions. The images are very dark, making it hard to distinguish objects or people.
--

: Use to brighten the image and make details more visible.
--

: Load a low-light image or darkened grayscale image. Apply different to the image with values like 0.5, 0.8, and 1.5. Observe the results. Experiment with changing the contrast by multiplying the pixel values by a constant. For example, try scaling the pixel values by 1.2, 1.5, and 2.0. Compare the original and enhanced images to see if details are more visible.

The code

<pre>import cv2 import numpy as np import matplotlib.pyplot as plt def zoom_image(img, zoom_factor): # img: Input image matrix (grayscale or RGB) # zoom_factor: Factor by which the image is zoomed # Get the size of the original image rows, cols, channels = img.shape # Initialize the zoomed image matrix zoomed_image = np.zeros((zoom_factor * rows, zoom_factor * cols, channels), dtype=np.uint8) # Apply pixel replication for i in range(rows): for j in range(cols): zoomed_image[i * zoom_factor:(i + 1) * zoom_factor, j * zoom_factor:(j + 1) * zoom_factor, :] = img[i, j, :] return zoomed_image # Load the image img = cv2.imread('Dahua-IPC-HDW5231R-Z-with-internal-IR-off- and-no-extra-light.jpg')</pre>

```

# Zoom factor (e.g., zoom by a factor of 2, you can change this to 3, 4,
etc.)
zoom_factor = 2
# Zoom the image
zoomed_img = zoom_image(img, zoom_factor)
# Display the original and zoomed image
plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.title('Original Image')
plt.axis('off')
plt.subplot(1, 2, 2)
plt.imshow(cv2.cvtColor(zoomed_img, cv2.COLOR_BGR2RGB))
plt.title(f'Zoomed Image by factor of {zoom_factor}')
plt.axis('off')
plt.show()

```

The results (Screenshot)



Lab Task No 2:

Solution:

Brief description (3-5 lines)

: You're working with a medical imaging system that captures X-ray images. The images often contain regions that are too bright or too dark, making it hard to analyze certain details.

: Use to balance the contrast, so the entire image is more evenly illuminated.

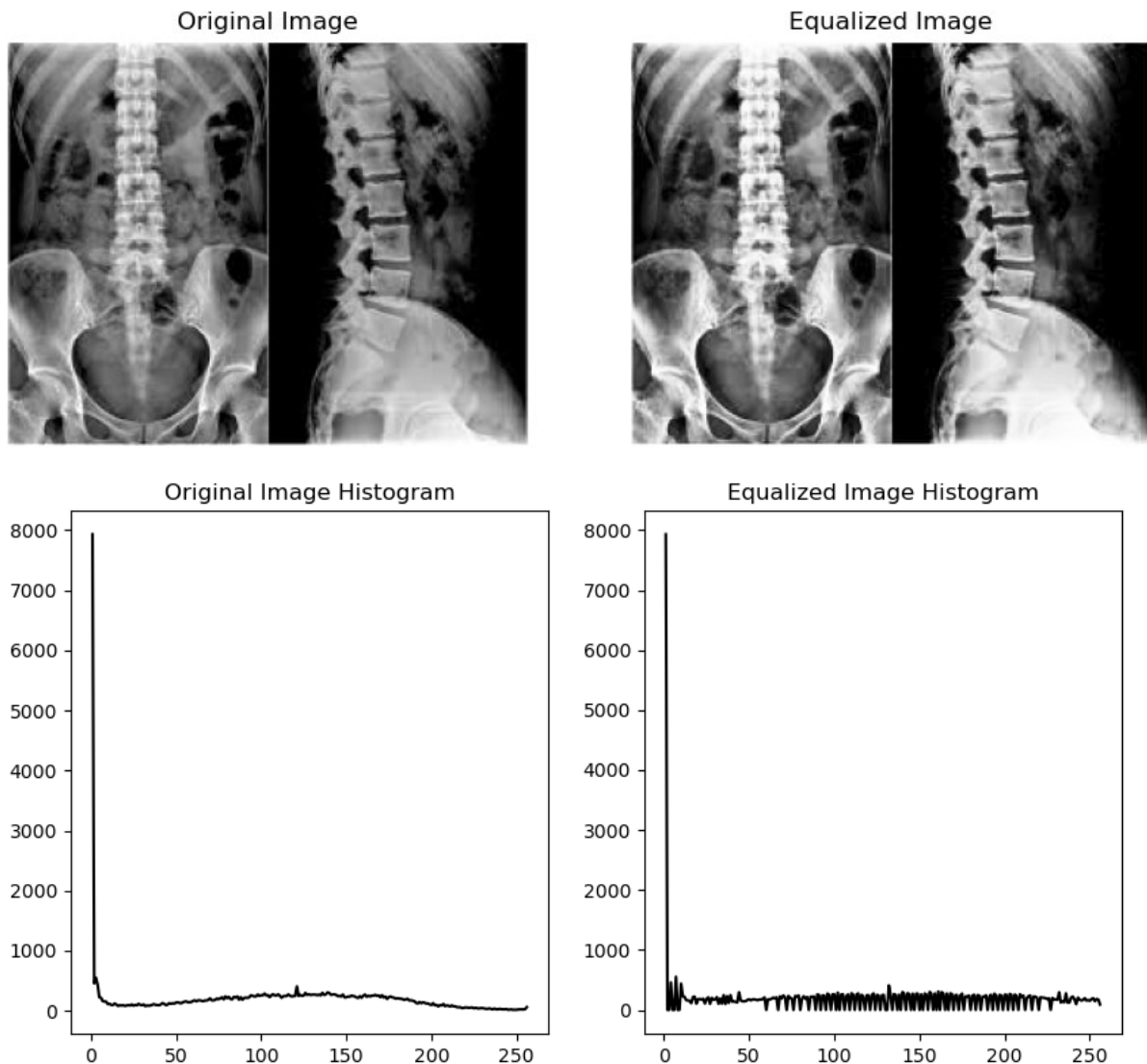
:Load a sample X-ray or CT scan image in grayscale. Apply histogram equalization using OpenCV's `cv2.equalizeHist()` function to improve the visibility of all areas. Display the original image and the histogram-equalized image side by side. Compare the histograms of both images to understand how the pixel intensities are redistributed.

The code

```
import cv2
import matplotlib.pyplot as plt
import numpy as np
# Load the image
image = cv2.imread('x-ray.jpg', cv2.IMREAD_GRAYSCALE)
# Apply histogram equalization
equalized_image = cv2.equalizeHist(image)
# Calculate histograms using numpy
hist_original, bin_edges_original = np.histogram(image.flatten(),
bins=256, range=[0, 256])
hist_equalized, bin_edges_equalized =
np.histogram(equalized_image.flatten(), bins=256, range=[0, 256])
# Display the original and equalized images
plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
plt.imshow(image, cmap='gray')
plt.title('Original Image')
plt.axis('off')
plt.subplot(1, 2, 2)
plt.imshow(equalized_image, cmap='gray')
plt.title('Equalized Image')
plt.axis('off')
plt.show()
# Plot the histograms
plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
plt.plot(bin_edges_original[1:], hist_original, color='black')
plt.title('Original Image Histogram')
plt.subplot(1, 2, 2)
```

```
plt.plot(bin_edges_equalized[1:], hist_equalized, color='black')  
plt.title('Equalized Image Histogram')  
plt.show()
```

The results (Screenshot)



Conclusion

In conclusion, Gray Level Transformations and Histogram Equalization are techniques that improve image contrast by redistributing pixel intensity values across the spectrum. This enhances visibility of details in dark or overly bright images. Histogram equalization adjusts the cumulative distribution function to achieve a more uniform histogram, stretching intensity values to cover the entire

spectrum. However, this technique may amplify noise and produce unrealistic effects in well-distributed images.